

Advanced Backend .NET. Весна 2026

Введение

В данном задании необходимо реализовать систему личного кабинета абитуриента. Сервис предусматривает распределенную архитектуру, некоторую ролевую модель и уведомления. О преимуществах и недостатках такого подхода мы поговорим на лекциях и практических занятиях. Каждый компонент в такой организации - это отдельное приложение, которое выполняет возложенные на него функции в рамках всей программной системы.

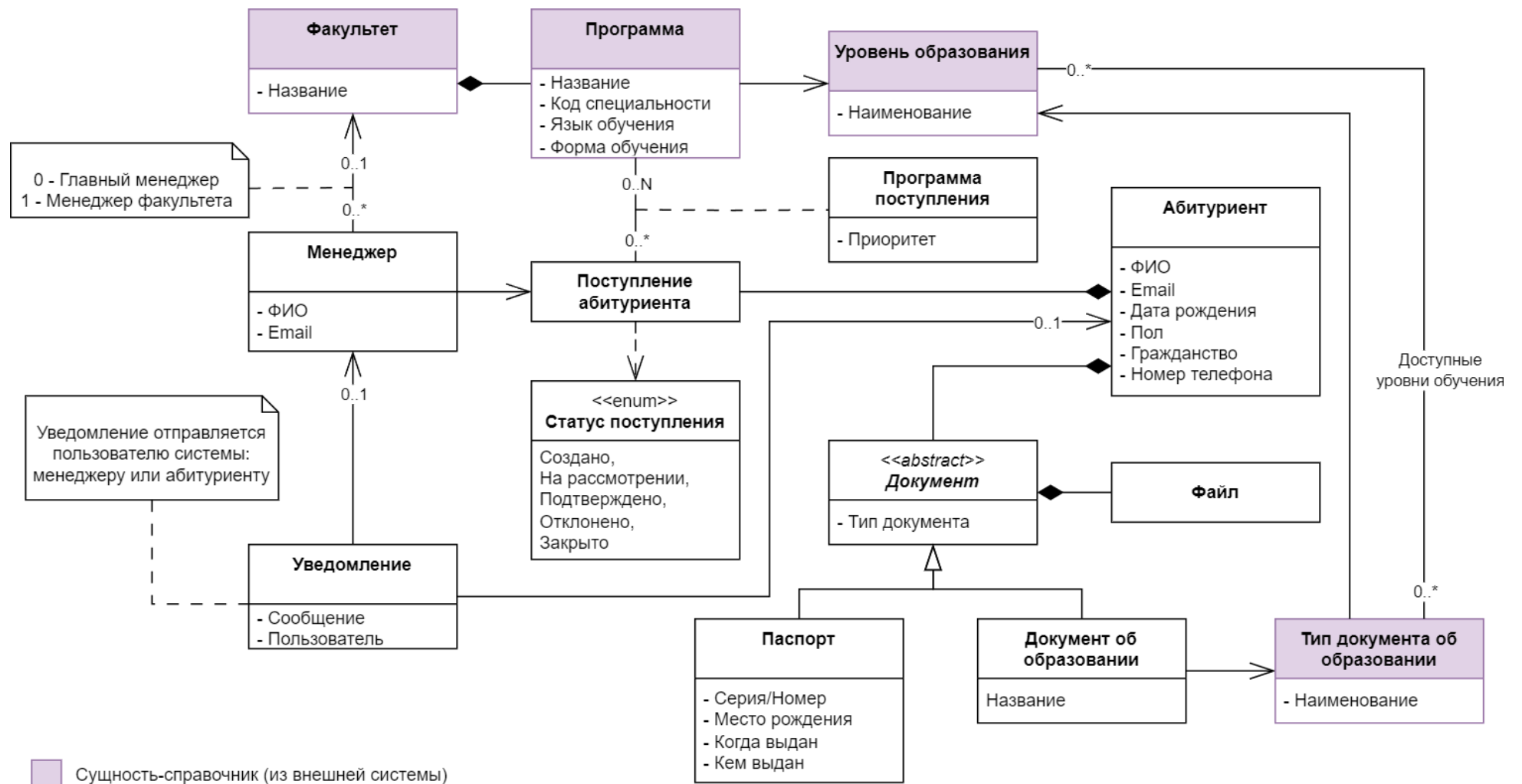
Описание классов

Для понимания предметной области — ниже представлена диаграмма классов, описывающая основные сущности.

Обратите внимание! На диаграмме (диаграммах) отражен предполагаемый объем для выполнения в течение курса. Любые сущности могут быть вами расширены при необходимости, могут быть добавлены новые. Важно, чтобы ваша система после внесенных изменений соответствовала установленным требованиям.

Пояснения к диаграмме

- **Абитуриент** – класс, содержащий основные данные об абитуриенте.
- **Поступление абитуриента** – класс, который содержит всю необходимую информацию для осуществления поступления (данная сущность представляет собой заявление на поступление). Класс содержит атрибут “Статус”, который может принимать следующие значения:
 - *Создано* - абитуриент начал поступление, вносит данные.
 - *На рассмотрении* - менеджер добавил поступление к себе в отслеживаемые для проверки данных.
 - *Подтверждено* - данные корректны, менеджер подтвердил это.
 - *Отклонено* - менеджер проверил данные, данные заполнены некорректно или не полностью.
 - *Закрыто* - доступ к редактированию данных закрыт. При выставлении данного статуса абитуриент больше не может вносить никакие изменения.
- **Документ** – абстрактный класс, предназначенный для содержания информации о каком-либо персональном документе абитуриента. Как можно заметить, наследниками данного класса являются:
 - **Паспорт** – класс, содержащий информацию о паспорте или другом официальном документе, удостоверяющем личность.
 - **Документ об образовании** – класс, который содержит информации об имеющемся абитуриента образовании.
- **Файл** – класс, который содержит данные о скане или копии какого-либо из документов абитуриента (метаданные, путь к этому файлу на диске и т.д.). Данная сущность обязательно имеет ссылку на документ, к которому принадлежит отсканированный файл .



- **Менеджер** – класс, указывающий на сотрудника вуза, занимающегося модерированием процесса поступления. В системе предусмотрено 2 типа менеджеров:
 - *Менеджер факультета* - сотрудник, работающий в приемной кампании конкретного факультета и отвечающий за абитуриентов, поступающих на программу данного факультета.
 - *Главный менеджер* - менеджер основной приёмной кампании, который помимо работы с абитуриентами отслеживают течение приёмной кампании.
- **Факультет** – класс, который содержит данные о факультетах вуза. Актуальные данные приходят из внешней системы.
- **Программа** – класс, содержащий информацию об образовательных программах вуза, открытых для поступления абитуриентов. Актуальные данные приходят из внешней системы.
- **Программа поступления** – класс-ассоциация, который содержит данные о выбранной абитуриентом программе для поступления.
- **Уровень образования** – класс, содержащий данные об уровнях образования.
- **Тип документа об образовании** - класс, содержащий данные о доступных типах документах об образовании. Класс содержит две связи с классом “Уровень образования”:
 - первая связь означает, к какому уровню образования относится данный тип документа;
 - вторая связь необходима для реализации логики хранения уровней обучения доступных для дальнейшего освоения (например, для уровня образования “Магистратура” следующим доступным уровнем будет являться “Аспирантура”). Актуальные данные приходят из внешней системы.

Доступ к внешней системе

Документация API - <https://1c-mockup.kreosoft.space/swagger/index.html>

- Логин: student
- Пароль: ny6gQnyn4ecbBrP9l1Fz

Минимальный функционал

Функционал клиента (абитуриента)

1. Возможность регистрации в системе
2. Возможность аутентификации/авторизации в системе

Рекомендуем при реализации аутентификации использовать связку *access-refresh токенов*.

3. Возможность смены пароля
4. Просмотр личного профиля
5. Обновление личных данных (ФИО, email, номер телефона, дата рождения, пол, гражданство)

6. Просмотр документов (отдельно каждого вида)
7. Возможность скачать скан документа
8. Возможность редактировать данные документа
9. Возможность удалить скан
10. Возможность загрузить новый скан документа
11. Возможность посмотреть список программ с возможностью пагинации и следующими фильтрациями:
 - a. фильтрация по факультету;
 - b. фильтрация по уровню образования;
 - c. фильтрация по форме обучения;
 - d. фильтрация по языку обучения;
 - e. поиск по названию/коду программы (по части).
12. Возможность добавить программу в список выбранных программ для обучения
Обратите внимание!
 - Количество выбранных программ не может быть больше, чем N (N - конфигурируется в приложении).
 - Программы должны относиться к одной ступени обучения. Например:
 - абитуриент “А” не может одновременно выбрать программу с уровнем “Магистратура” и “Аспирантура”;
 - абитуриент “В” может выбрать программу с уровнем “Специалитет”, если у него уже выбрана программа с уровнем “Бакалавриат”.
 - Если у абитуриента добавлен документ об образовании, уровень выбранной программы должен быть либо аналогичен уровню документа об образовании, либо входить в список доступных для обучения
13. Возможность изменить приоритет программы
14. Возможность удалить программу из выбранного списка
15. Получить уведомление на email об изменении статуса поступления

Обратите внимание! Абитуриент не может редактировать данные (личные данные, документы, сканы, программы), если статус у его поступления “Закрето”.

Функционал персонала

В системе предусмотрено 3 роли пользователя помимо абитуриента:

- **Менеджер** - сотрудник факультета, основной задачей которого является работа с абитуриентами и их документами.
- **Главный менеджер** - помимо функционала актера с ролью “Менеджер” имеют возможность управлять течением приёмной кампании.
- **Администратор** - основной задачей является управление и обслуживание системы. Кроме своих возможностей, он обладает всеми системными функциями пользователей с другими ролями.

Функционал “Менеджера”

1. Возможность аутентификации/авторизации в системе
2. Возможность изменить данные для входа (email, пароль)
3. Взять поступление абитуриента.
4. Отказаться от поступления абитуриента (вернуть его в общий пул заявок).

5. Просмотреть заявки абитуриентов с пагинацией и следующими фильтрациями и сортировками:
 - a. Поиск по части имени;
 - b. Фильтрация по программе;
 - c. Фильтрация по факультету - multiselect (у абитуриента должна быть выбрана хотя бы одна программа данного факультета);
 - d. Фильтрация по статусу поступления;
 - e. Отображение только тех абитуриентов, у которых еще не назначен менеджер;
 - f. Отображение абитуриентов, привязанных к данному менеджеру;
 - g. Сортировка по дате внесения последних изменений (по убыванию, по возрастанию).
6. Посмотреть данные абитуриента:
 - a. Личные данные ФИО, email, номер телефона, дата рождения, пол, гражданство)
 - b. Документы (отдельно каждого вида) с возможностью посмотреть/скачать сканы документа
 - c. Выбранный программы
7. Обновление личных данных абитуриента
8. Возможность редактировать данные документа абитуриента
9. Возможность удалить скан документа абитуриента
10. Возможность загрузить новый скан документа абитуриента
11. Возможность изменить приоритет программы
12. Возможность удалить программу из выбранного списка
13. Изменить статус поступления

Обратите внимание! Менеджер может просматривать данные всех абитуриентов, но редактировать только тех абитуриентов, для которых он назначен менеджером.

Функционал “Главного менеджера”

1. Возможность посмотреть список менеджеров, главных менеджеров
2. Возможность назначить менеджера на поступление, если оно свободно

Примечание: при назначении менеджера, абитуриенту и менеджеру должно быть отправлено уведомление об этом.

Обратите внимание! Главный менеджер и администратор могут просматривать и редактировать данные всех абитуриентов.

Функционал “Администратора”

1. Возможность импорта справочников: факультет, программа, уровень образования
2. Возможность посмотреть статус импорта справочников
3. Возможность создать нового менеджера, главного менеджера

Примечание: при создании менеджера на указанный электронный адрес должно быть отправлено уведомление.

Для тестирования функционала отправки email может быть использован MailDev (<https://github.com/maildev/maildev>), либо можете использовать настоящий SMTP-сервер.

При выборе иных технологий рекомендуем проконсультироваться и утвердить их с преподавателями направления.

4. Возможность отредактировать данные менеджера
5. Возможность удалить менеджера, главного менеджера

Задание

1. Спроектировать архитектуру системы, подготовить артефакты. Артефакты, полученные в результате проектирования, обязательно необходимо push'ить в ваш git! Результирующий набор артефактов должен в полной мере описывать ваш проект системы. От архитектуры всей системы до каких-то важных деталей реализации отдельных сервисов.

Примечание: для работы с артефактами рекомендуем использовать draw.io (<https://app.diagrams.net/>). Он формирует файлы в формате xml, который отлично подходит для git.

2. Разработать спецификацию API сервисов. Спецификация API должна быть представлена в виде swagger.json или Markdown файлом и сохранена в git.
3. Проработать структуру вашего решения. Какие проекты будут созданы, какие будут исполняемыми, какие будут библиотеками классов, связи между ними.
4. Разработать описанные компоненты (версия платформы для всех компонентов не ниже .Net 8)
5. *Разработать unit-тесты.
6. *Разработать нагрузочные тесты.

*Примечание: пункты, отмеченные символом * - рекомендованы к выполнению, но не являются обязательными.*

Бонусные пункты

Информация, необходимая для выполнения перечисленных ниже пунктов, не будет рассматриваться на лекциях.

По всем бонусным пунктам общаемся в индивидуальном порядке, на консультациях, при условии, что у студента нет проблем с реализацией основного задания.

1. Редактировать данные абитуриента может любой менеджер факультета, где у абитуриента первый приоритет поступления. Условие задания по ссылке  Задание .NET - ЛК Абитуриента изменяется при выполнении этого пункта.
2. Настроена инфраструктура сбора логов из каждого приложения в вашей системе. Сбор логов в одном хранилище. Возможность просмотра логов всех приложений в одном месте.
3. Возможность трассировки запросов в системе. Трассировка распределенных запросов. Возможность просмотра трейсов в одном месте.
4. Возможность просмотра истории изменений и автора этих изменений для данных абитуриента и его поступления.
5. Дополнительный канал уведомлений в режиме реального времени через websocket.

Ссылка на диаграммы

Доступ к диаграммам будет предоставлен после КТ1

<https://app.diagrams.net/#G1eTXeXZn8BJoADOmRVL2N2lrR-BgCxpGu#%7B%22pageId%22%3A%22XJuEihj2ZfyuyvtKoxH3%22%7D>

Обратите внимание! В файле с диаграммами 4 листа.

Правила дисциплины

Обратите внимание (распространяется на все КТ)!

- Общение между студентами в рамках выполнения работы не запрещено, но при выявлении “списывания” - баллы будут обнулены.
- В рамках выполнения работы студентам не запрещено пользоваться различными источниками при условии понимания используемых материалов. В случае, если студент не может объяснить принятое решение при разработке/проектировании, либо не понимает как работают использованные им инструменты, это будет приравниваться к списыванию.
- Все этапы работы сдаются студентами индивидуально, общение в рамках сдачи с другими студентами не допускается.
- Все разделы задания могут быть дополнены и изменены по согласованию преподавателей и студентов.

Оценивание

На первой и второй контрольных точках будет необходимо показать ваш прогресс по текущему заданию. Предполагается зачетно-штрафная система:

- задание выполнено полностью и качественно (реализован необходимый минимум для сдачи задания см. ниже) -> штрафов не предусмотрено;
- за частичную реализацию или её отсутствие -> штраф 0,25-1 балл.

Формат сдачи первой и второй КТ: перед КТ будет создано задание в Google Classroom, вам будет необходимо до начала КТ - прикрепить к нему ссылку на ваш репозиторий, а также отметить коммит/коммиты, в которых хранятся подготовленные вами артефакты.

Во время сдачи необходимо показать ваш текущий прогресс по той версии артефактов, которые у вас прикреплены в Google Classroom.

Обратите внимание! Незакоммиченные изменения, либо сделанные после начала КТ, проверяться не будут.

Реализация больше минимума приветствуется и поощряется хорошим настроением преподавателей! 😊

КТ 1

Необходимый минимум для сдачи 1КТ

1. Спроектировать систему

Примечание: система должна быть распределенной и масштабируемой в

*будущем. Один из сервисов должен быть приложением ASP.NET Core MVC .
Как минимум один из сервисов должен быть приложением Web Api.*

2. Создать артефакты, описывающие архитектуру

Обратите внимание! Артефакты должны быть обязательно сохранены в *git*'е в вашем решении.

3. Разработать спецификацию API сервисов.

Настоятельно рекомендуем при проектировании использовать принципы REST. Например, в качестве справки <https://blog.octo.com/design-a-rest-api> и <https://stackoverflow.blog/2020/03/02/best-practices-for-rest-api-design/>

Обратите внимание! Спецификация API должна быть представлена в виде swagger.json или Markdown файлом и сохранена в git.

По итогу 1 КТ для каждого студента будет зафиксирована архитектура, следуя которой будет необходимо разработать дальнейшее решение: предложенная студентом во время сдачи (может быть модифицирована группой преподавателей), либо предоставлена новая группой преподавателей, в случае, если задание не выполнено, или проект детализирован недостаточно.

КТ 2

Необходимый минимум для сдачи 2КТ

1. Проработать структуру решения. Solution создан, разделен на проекты, связи между частями логичные и достаточные, структура позволяет масштабировать решение в будущем.
2. Реализовать несколько компонентов (объем согласовывается на основе архитектуры студента).

КТ 3

Финальная сдача будет состоять из двух этапов:

1. Практико-теоретический экзамен
2. Проверка реализованного решения

Практико-теоретический экзамен

В рамках экзамена студенту будет предоставлено несколько вопросов, связанных с темами, которые были рассмотрены в рамках базового и продвинутого курсов, а также базовые знания на понимание платформы.

По результатам сдачи студенту будет выставлен коэффициент (от 0 до 1). На данный коэффициент будут умножены баллы полученные студентом за сдачу решения с учётом ранее полученных штрафов. Таким образом, итоговая формула оценивания выглядит следующим образом: $(S-F1-F2)*K$, где

S - итоговые баллы за реализованное решение,

F1 - штраф, полученный за 1КТ,

F2 - штраф, полученный за 2КТ,

K - коэффициент, полученный за практико-теоретический экзамен

Проверка реализованного решения

Будет описано позже – ближе к сдаче